DBT PORTFOLIO PROJECT

LAYER: INTERMEDIATE

DBT Intermediate Model

Moving Average | Window Functions | Time Series | Snowflake | Rolling Analytics

Project Summary

Computes 7-period and 30-period rolling moving averages of the healthcare claims metric value for each entity, partitioned by entity_id and ordered by period_date. Also tracks the actual row count in each window to flag warm-up periods at the start of each entity's time series. This model is the spine of the mart layer.

Thomas Foster

Senior Data Analyst | Snowflake | DBT | Healthcare Analytics

github.com/thomasfoster | thomasfoster79@yahoo.com

1 PROJECT OVERVIEW

Model Name:	int_moving_average
Layer:	Intermediate
Materialization:	table (referenced by mart as primary spine)
Source:	{{ source('healthcare', 'claims') }}
Key Outputs:	moving_avg_7, moving_avg_30, window_row_count per entity/date
Audience:	Mart layer (fct_claims_analytics), Tableau dashboards

2 BUSINESS PROBLEM

Raw daily or monthly claim metrics are noisy -- a single bad period (data lag, holiday, outlier claim batch) can make trends appear volatile and mislead decision-makers. Moving averages smooth that noise into actionable trend lines, letting finance and operations teams see the true direction of claim volumes and financial metrics without reacting to one-off spikes.

- 7-period MA (short): Captures near-term trends -- useful for weekly operational monitoring.
- 30-period MA (long): Captures sustained trends -- used in monthly finance reviews.
- window_row_count: Flags warm-up rows where fewer than 7 or 30 periods of history exist (start of series).
- Partitioning by entity_id ensures each provider/payer gets its own independent rolling window.

3 TECHNICAL EXPLANATION

- AVG() OVER (...ROWS BETWEEN 6 PRECEDING AND CURRENT ROW): 7-period window (6 prior + current).
- AVG() OVER (...ROWS BETWEEN 29 PRECEDING AND CURRENT ROW): 30-period window (29 prior + current).
- COUNT() OVER same window: Returns actual number of rows in window -- < 7 or < 30 means warm-up period.
- PARTITION BY entity_id: Each entity's window resets independently -- no bleed between entities.
- ORDER BY period_date: Ensures chronological ordering within each entity's partition.
- Materialized as a TABLE: Mart layer joins to this model repeatedly -- table avoids re-computation.

Key Techniques: AVG() OVER, COUNT() OVER, ROWS BETWEEN N PRECEDING AND CURRENT ROW, PARTITION BY

4 FULL MODEL CODE

```
{{ config(materialized='table') }}

select
    entity_id,
    payer,
    state,
    specialty,
    period_date,
    metric_value,

    avg(metric_value) over (
        partition by entity_id
        order by period_date
        rows between 6 preceding and current row
    ) as moving_avg_7,

    avg(metric_value) over (
        partition by entity_id
        order by period_date
        rows between 29 preceding and current row
    ) as moving_avg_30,

    count(metric_value) over (
        partition by entity_id
        order by period_date
        rows between 6 preceding and current row
    ) as window_row_count

from {{ source('healthcare', 'claims') }}
```

5 EXAMPLE OUTPUT

entity_id	period_date	metric_value	moving_avg_7	moving_avg_30	window_rows
E-001	2025-01-01	4250.75	4250.75	4250.75	1
E-001	2025-01-08	4380.20	4315.48	4315.48	2
E-001	2025-04-01	4620.10	4558.32	4401.22	7
E-001	2025-07-01	4890.55	4801.44	4612.38	30

** window_rows < 7 or < 30 indicates a warm-up period where full window history is not yet available.*